

Auditoría técnica del código

Revisión de seguridad, dominios de negocio y calidad del código de la aplicación (apps/web). Panorámica priorizada.

Alcance · apps/web (Next.js + PostgreSQL, BFF, multi-tenant)

Método · 4 pases de auditoría dirigida (seguridad · comercial/finanzas · operaciones/infra · arrendadores) **Fecha** · 23 jul 2026

Postura general: los cimientos de seguridad son sólidos (aislamiento por tenant con RLS *fail-closed* verificado por assert, SQL parametrizado, uploads validados por *magic bytes*, secretos server-only, CSRF, RBAC en todas las rutas). Los riesgos materiales se concentran en **condiciones de carrera de dinero** (duplicar facturas/campañas/reservas), en que el **candado de facturación digital es falsificable**, en **multi-moneda inexistente**, y en un **pipeline de deploy automatizado roto** (los deploys reales, manuales, sí aplican todo).



01 Arquitectura y stack

Producto vivo: el shell `/demo` (Next.js 14 App Router) con *Route Handlers* como BFF sobre PostgreSQL 16 (`db/schema.sql`), acceso con la librería `pg` y consultas parametrizadas. Multi-tenant con aislamiento por `tenant_id` vía Row-Level Security. Un `apps/api` (Fastify/Prisma) quedó archivado y no corre. Despliegue manual por SSH a un droplet (PM2, `spaces-web`), servido hoy por HTTP.

El estado del front se hidrata desde `GET /api/estado`, que agrega las *slices* de todos los módulos filtradas por permiso. La lógica de negocio derivada (P&L, disponibilidad, alertas, candado) vive en `lib/data/derive.ts`; la de escritura en `lib/server/*-repo.ts`.

02 Postura de seguridad — lo que está bien construido

- **RLS fail-closed + FORCE** en todas las tablas con `tenant_id`, con dos *ASSERT* en la migración: el rol de app no es superusuario/bypass, y no queda ninguna tabla de tenant sin cubrir. Sin el GUC de tenant no se ve ni escribe nada.
- **Tenant transaction-local** (`set_config(...,true)`): correcto para un pool que reúsa conexiones entre tenants; las escrituras además llevan `and tenant_id=$N` explícito.

- **SQL 100% parametrizado**; los `${...}` en queries son nombres de columna de allowlists o placeholders, nunca input de usuario.
- **Uploads** validados por *magic bytes* reales (no el MIME declarado), allowlist y tamaño; rechaza SVG con scripts.
- **Secretos server-only** (`DATABASE_URL`, `SPACE_EYE_*`, `RESEND`, Spaces): ninguno llega al bundle del cliente; el cliente solo ve `NEXT_PUBLIC_*`.
- **Auth**: bcrypt (cost 10), tokens de sesión de 256 bits, lecturas pre-sesión por funciones `SECURITY DEFINER` acotadas, invalidación de sesión en logout y reset.
- **CSRF** double-submit en toda mutación con sesión; **RBAC** con `exigir()` en todas las rutas; **candado del Dueño** sobre la mayoría de mutaciones de dinero.

03 Hallazgos priorizados

• ALTO Riesgos materiales

A-1 Duplicación por condición de carrera: facturas, campañas y reservas

COMERCIAL · FINANZAS

finanzas-repo.ts:137,157 · campanas-repo.ts:452 · campanas-repo.ts:335-372 · db/schema.sql:383,532

ESCENARIO

Los checks de "¿ya existe?" corren *antes* del `begin` y **faltan índices únicos** (`facturas.campana_id`, `campanas.propuesta_id`) y `FOR UPDATE` sobre el sitio al reservar. Dos peticiones concurrentes (doble clic, dos pestañas) generan 2 facturas + 2 cobranzas de la misma campaña, o 2 campañas de la misma propuesta, o dos reservas del mismo slot/fechas → doble cobro, doble booking, sobreventa (13/12).

RECOMENDACIÓN

Índices `UNIQUE` en `facturas.campana_id` y `campanas.propuesta_id`; al reservar, `SELECT ... FOR UPDATE` del sitio o una restricción de exclusión por rango de fechas; descontar el spot atómicamente.

A-2 El candado de facturación digital es falsificable

COMERCIAL · FINANZAS

playlogs-repo.ts:85-96 · ot-repo.ts:204-208

ESCENARIO

(1) Un proof-of-play **vacío** (0 reproducciones) enciende `fotos_comprobatorias=true` con solo `r.ok` → una campaña que nunca salió al aire puede facturarse. (2) En una campaña **híbrida**, cerrar UNA OT de montaje de la parte fija enciende a la vez `fotos_comprobatorias` y `reporte_publicacion` para toda la campaña, saltándose el proof-of-play de las pantallas digitales.

RECOMENDACIÓN

Exigir `vacio === false` (reproducciones reales) para encender el flag digital; en híbridas, calcular el candado por segmento (la parte digital exige su propio proof-of-play, la fija su OT).

A-3 Multi-moneda inexistente: la factura sale siempre en soles

FINANZAS · ARRENDADORES

finanzas-repo.ts:162 · db/schema.sql:366,571 · derive.ts:234,264-276,532-550

ESCENARIO

El INSERT de factura fija el literal 'PEN' e ignora `campanas.moneda`; la campaña nace con moneda default 'PEN' aunque la organización es MXN (RFC/CFDI/IVA 16%). El P&L, el "por cobrar" y el costo de renta suman importes de distintas divisas **sin convertir**. Hoy queda oculto porque todo cae a PEN, pero cualquier contrato/campaña en otra moneda se suma 1:1.

RECOMENDACIÓN

Propagar la moneda del tenant a campaña/reserva/factura/contrato; sumar y facturar en la divisa correcta; si se admiten mixtas, convertir con un tipo de cambio antes de totalizar.

A- 4 Editar la cuenta bancaria del arrendador no pasa por el candado del Dueño

SEGURIDAD ·
ARRENDADORES

app/api/arrendadores/[id]/route.ts:12-13 · arrendadores-repo.ts:683-684

ESCENARIO

El PATCH del arrendador usa solo `exigir('arrendadores', 'crear')` —sin `exigirCambioSensible`— pero permite cambiar `cuenta_bancaria` y `forma_pago`. En cambio, *borrar* el arrendador (menos peligroso) sí exige candado. Un usuario sin desbloquear cambia la CLABE a la suya y los pagos de renta subsiguientes se redirigen — justo el movimiento de dinero que el candado existe para frenar.

RECOMENDACIÓN

Enrutar la edición de datos bancarios por `exigirCambioSensible` (mismo candado que pagos/contratos).

A-5 Pipeline de deploy automatizado roto (lockfile + migraciones)

INFRAESTRUCTURA

package-lock.json (postcss 8.4.31 vs ^8.4.49) · .github/workflows/deploy.yml:24,47,53-56

ESCENARIO

El `package-lock.json` quedó desincronizado (postcss), así que `npm ci` aborta; y `deploy.yml` solo aplica 4 migraciones viejas de `20260706` —faltan ~20 posteriores (RLS Hardening-1, almacén, password_resets, pausa legal...)—. Si se usara ese pipeline, prod quedaría sin tablas y, p. ej., cerrar una OT de desmontaje fallaría por completo (la tabla del almacén no existiría).

RECOMENDACIÓN

Regenerar y commitear el lockfile; hacer que el deploy aplique **todas** las migraciones pendientes (p. ej. iterando el directorio, no una lista fija). **Nota:** los deploys reales son *manuales* y ya aplicaron todas las migraciones — esto es deuda de automatización, no un fallo del prod actual.

M-1 recalcularPresupuesto corrompe la base de facturación de campañas de propuesta COMERCIAL

campanas-repo.ts:27-43,514,522-540,584,603

ESCENARIO

La campaña de propuesta fija `presupuesto_neto = snap.base` a propósito. Pero *extender fecha o confirmar reserva* llaman `recalcularPresupuesto`, que hace $\Sigma(\text{precio} \times \text{días}/30)$ sobre el precio ya *neto de comisión* → sobrescribe la base con una suma prorrateada menor. La factura posterior cobra menos que el paquete aceptado.

RECOMENDACIÓN

No recalcular el presupuesto de campañas nacidas de propuesta (respetar el snapshot), o recalcular contra el bruto, no el neto de comisión.

M-2 Disponibilidad digital inconsistente; los spots no se decrementan al vender

COMERCIAL

campanas-repo.ts:360-371,207-216 · derive.ts:944-947 · confirmarReserva:578

ESCENARIO

El booking mide 1 campaña = 1 slot; la rejilla mide $\Sigma \text{spotsReservados}$. Además `spots_disponibles` nunca se resta al vender (solo se suma al liberar), así que la columna puede mostrar "12/12" con pantallas ya vendidas y `confirmarReserva` nunca marca OCUPADO.

RECOMENDACIÓN

Unificar el modelo de conteo de spots y decrementar `spots_disponibles` atómicamente al confirmar; derivar la disponibilidad de una sola fuente.

M-3 El total mostrado de la propuesta ≠ la base facturada (aprobación parcial)

COMERCIAL

propuestas-repo.ts:52-101 vs 112-160,223

ESCENARIO

El total del listado/liga pública se calcula sobre **todos** los items; el snapshot y la campaña, sobre los **aprobados**. Con aprobación granular, el cliente ve un total que incluye pantallas no aprobadas y difiere de lo que se factura.

RECOMENDACIÓN

Mostrar el total sobre los items aprobados (o ambos, claramente etiquetados).

M-4 /api/estado transporta base64 pesados en cada carga

INFRAESTRUCTURA

arrendadores-repo.ts:125,174-176 · campanas-repo.ts:114-122 · ordenes-compra-repo.ts:25-33 · ot-repo.ts:39-49 · config-repo.ts:24

ESCENARIO

Documentos de contrato, creativos (imagen/video/HTML), OC y evidencias viajan como base64 dentro de `/api/estado` en cada GET. Con decenas de registros, cada carga del dashboard descarga MBs e infla la memoria del store. Los adjuntos de pagos sí se excluyeron a propósito; falta la misma técnica aquí.

RECOMENDACIÓN

Excluir los base64 del estado global y servirlos perezosamente por ruta/URL firmada, como ya se hace con los adjuntos de pago.

M-5 Escrituras en cada GET de /api/estado; recálculo sin throttle

INFRAESTRUCTURA

estado/route.ts:58-64 · arrendadores-repo.ts:206-227

ESCENARIO

Cada carga dispara hasta 4 barridos que escriben. Son idempotentes por guardas, pero `recomputarEstatusArrendadores` (introducido en esta sesión para el recálculo de estatus) lanza 2 UPDATE con *scan* de tabla en cada carga. Con polling o varias pestañas, multiplica transacciones y contención de locks.

RECOMENDACIÓN

Acotar el recálculo a "a lo más 1× cada N minutos por tenant" (marca de tiempo), o moverlo a un job programado.

M-6 La publicación en DOOHmain bloquea el request de validación

OPERACIONES

app/api/campanas/[id]/validar/route.ts:40 · doohmain.ts:160,283-311

ESCENARIO

Publicar corre un subproceso Python **secuencial por cada creativo × pantalla** (timeout 120s c/u), todo dentro del POST de validar. Una campaña con muchas pantallas puede tardar minutos y superar el timeout del gateway/PM2, dejando la publicación a medias.

RECOMENDACIÓN

Mover la publicación a una cola/tarea en segundo plano; responder rápido y reflejar el progreso.

M-7 Rate-limit en memoria y confiando en X-Forwarded-For

SEGURIDAD

lib/server/rate-limit.ts:13,32-35

ESCENARIO

El límite anti-fuerza-bruta (login/signup/forgot/reset) es un Map por proceso: se reinicia en cada deploy y no sirve con múltiples instancias. Además toma el primer hop de `X-Forwarded-For`; si la app es alcanzable directo o el proxy no reescribe el header, rotarlo da intentos ilimitados.

RECOMENDACIÓN

Mover el contador a un store compartido (Redis) y anclar la clave a una IP de confianza (`x-real-ip` del proxy), no al XFF crudo.

M-8 config_negocio es una fila global sin tenant_id

SEGURIDAD

lib/server/config-repo.ts:32 · tenant.ts:63-64

ESCENARIO

Moneda, IVA(s) y loop/slot viven en una fila global sin `tenant_id` (fuera del assert de cobertura RLS). Un admin de cualquier organización los edita **para todas** — no rompe confidencialidad, pero sí la integridad cross-tenant de los cálculos económicos. La identidad (razón social) ya se migró a `tenants`; falta moneda/IVA/loop.

RECOMENDACIÓN

Mover estos campos a `tenants` (o una tabla config por tenant con RLS).

M-9 Barridos no atómicos → notificaciones duplicadas

OPERACIONES

ot-repo.ts:62-90 · finanzas-repo.ts:80-97

ESCENARIO

`notificarOTsVencidas` y `recordarCobranzasVencidas` hacen `SELECT ... NOT EXISTS` y luego insertan en sentencias separadas, sin lock. Dos GET concurrentes (StrictMode, varias pestañas) insertan la misma alerta dos veces.

RECOMENDACIÓN

Índice único sobre la notificación (p. ej. por `link` /entidad) o insertar con `ON CONFLICT DO NOTHING`.

• BAJO Deuda y robustez

ID	HALLAZGO	UBICACIÓN
B-1	Flujos multi- <code>q()</code> no atómicos (cobranza→factura PAGADA; activo→movimiento de almacén): si el 2º paso falla, quedan inconsistentes. Usar <code>withTenantTx</code> .	<code>finanzas-repo.ts:194-198</code> · <code>almacen-repo.ts:53-62, 84-94</code>
B-2	El permiso de <i>lectura</i> (<code>ver</code>) dispara escrituras (barridos): un rol auditor provoca UPDATE/INSERT al abrir el dashboard.	<code>estado/route.ts:58-64</code>
B-3	Errores no- <code>AppError</code> /no-PG se enmascaran como 500 "Error interno" sin ID de request ni logger estructurado; <code>FacturaError</code> / <code>PropuestaError</code> no extienden <code>AppError</code> . Dificulta el diagnóstico.	<code>errores.ts:143-147</code>
B-4	La OC no se reconcilia con el monto facturado y <code>oc_recibida</code> puede re-marcarse N veces (sin dedupe).	<code>ordenes-compra-repo.ts:53, 65</code>
B-5	La cookie de override de tenant usa <code>COOKIE_SECURE==='1'</code> en vez de <code>cookieSecure()</code> → puede ir sin <code>Secure</code> .	<code>app/api/tenant-activo/route.ts:23</code>
B-6	<code>doohmain.ts</code> sin <code>import 'server-only'</code> (defensa en profundidad; no contiene secretos).	<code>doohmain.ts:1-3</code>
B-7	Cambiar la contraseña desde <code>/api/perfil</code> no cierra las demás sesiones (el reset sí). Una sesión comprometida sobrevive.	<code>perfil-controller.ts</code>
B-8	Coerciones a \$0 silenciosas: tarifa faltante → reserva \$0; comisión 100% → neto \$0 sin guardarraíl.	<code>campanas-repo.ts:328</code> · <code>propuestas-repo.ts:58, 534</code>
B-9	Sin <code>CHECK</code> en BD (<code>fecha_fin>inicio</code> , <code>monto>0</code>): la integridad depende de pasar por el controller. Renta 0 permitida.	<code>db/schema.sql</code> (<code>contratos_arrendamiento</code>)
B-10	~25 PDFs y decenas de PNG versionados en el repo: inflan el clon y el checkout de cada deploy. Mover a almacenamiento externo/LFS.	<code>raíz</code> · <code>docs/</code> · <code>manual-shots/</code>
B-11	<code>almacen_activos.etiqueta</code> sin unicidad: dos retiros del mismo sitio generan etiquetas repetidas.	<code>ot-repo.ts:217</code>

04 Resuelto durante esta sesión

-  **Estatus de contratos y pagos "congelado"** (era ALTO): falseaba el costo de renta y silenciaba alertas. Ahora se recalcula contra la fecha de hoy (introdujo M-5, acotable).
-  **cerrarOT devolvía 500** aunque cerraba: leía la OT tras el commit, cuando la RLS ya no tenía tenant. Se corrigió (lee dentro de la transacción). Se verificó que el patrón no está en ningún otro repo.

05 Plan de remediación sugerido

1. **Blindar el dinero (A-1, A-4):** índices `UNIQUE` + `FOR UPDATE` en factura/campaña/reserva, y candado en la cuenta bancaria. Es lo de mayor impacto y menor esfuerzo.
2. **Verdad del candado digital (A-2):** exigir reproducciones reales y calcular el candado por segmento en híbridas.
3. **Moneda (A-3):** propagar la moneda del tenant end-to-end y facturar en ella.
4. **Automatizar el deploy (A-5):** regenerar el lockfile y aplicar todas las migraciones en el pipeline.
5. **Aligerar /api/estado (M-4, M-5):** sacar los base64 del estado y acotar los barridos.
6. **Endurecer anti-abuso (M-7, M-8):** rate-limit compartido y config económica por tenant.